

User Profiles & Social Features

▼ Relevant source files

Backend/STUDER/src/main/java/facu/studer/DTOs/notification/NotificationResponseDTO.java

Backend/STUDER/src/main/java/facu/studer/DTOs/notification/UserNotificationResponseDTO.java

Backend/STUDER/src/main/java/facu/studer/DTOs/user/FriendStatusResponseDTO.java

Backend/STUDER/src/main/java/facu/studer/DTOs/user/UserPublicResponseDTO.java

Backend/STUDER/src/main/java/facu/studer/DTOs/user/UserSearchPageResponseDTO.java

Frontend/STUDER/src/app/features/search/search.component.html

Frontend/STUDER/src/app/features/search/search.component.ts

Frontend/STUDER/src/app/features/users/components/profile/profile.component.html

Frontend/STUDER/src/app/features/users/components/profile/profile.component.ts

Frontend/STUDER/src/app/features/users/components/user-profile/user-profile.component.html

Frontend/STUDER/src/app/features/users/components/user-profile/user-profile.component.ts

Frontend/STUDER/src/app/features/users/friend.model.ts

Frontend/STUDER/src/app/features/users/friend.service.ts

Frontend/STUDER/src/app/features/users/user.model.ts

Frontend/STUDER/src/app/features/users/user.service.ts

This section covers the user-centric features of STUDER, including profile management, the social graph (following and friendship), and user discovery through search.

Overview

The user profile system differentiates between a user's own account management and the public view of other users. The social layer utilizes a "two-flag" model (`isFollowing` and `isFriend`) to manage relationships, which gates specific features like direct messaging.

Data Models & Interfaces

The frontend utilizes several interfaces to represent user states and relationship statuses.

Interface	Purpose	File
<code>User</code>	Full profile data for the authenticated user (includes email).	Frontend/STUDER/src/app/features/users/user.model.ts 32-42
<code>UserPublic</code>	Publicly visible data for other users.	Frontend/STUDER/src/app/features/users/user.model.ts 44-53
<code>UserUpdateRequestDTO</code>	Payload for updating email or password.	Frontend/STUDER/src/app/features/users/user.model.ts 27-30
<code>FriendStatus</code>	Tracks if the current user follows or is friends with another.	Frontend/STUDER/src/app/features/users/friend.model.ts 1-4

Sources: Frontend/STUDER/src/app/features/users/user.model.ts 1-60

Frontend/STUDER/src/app/features/users/friend.model.ts 1-15

User Management Service

The `UserService` handles all interactions with the `/api/v1/users` endpoints. It supports profile retrieval, searching, and multipart updates for profile pictures.

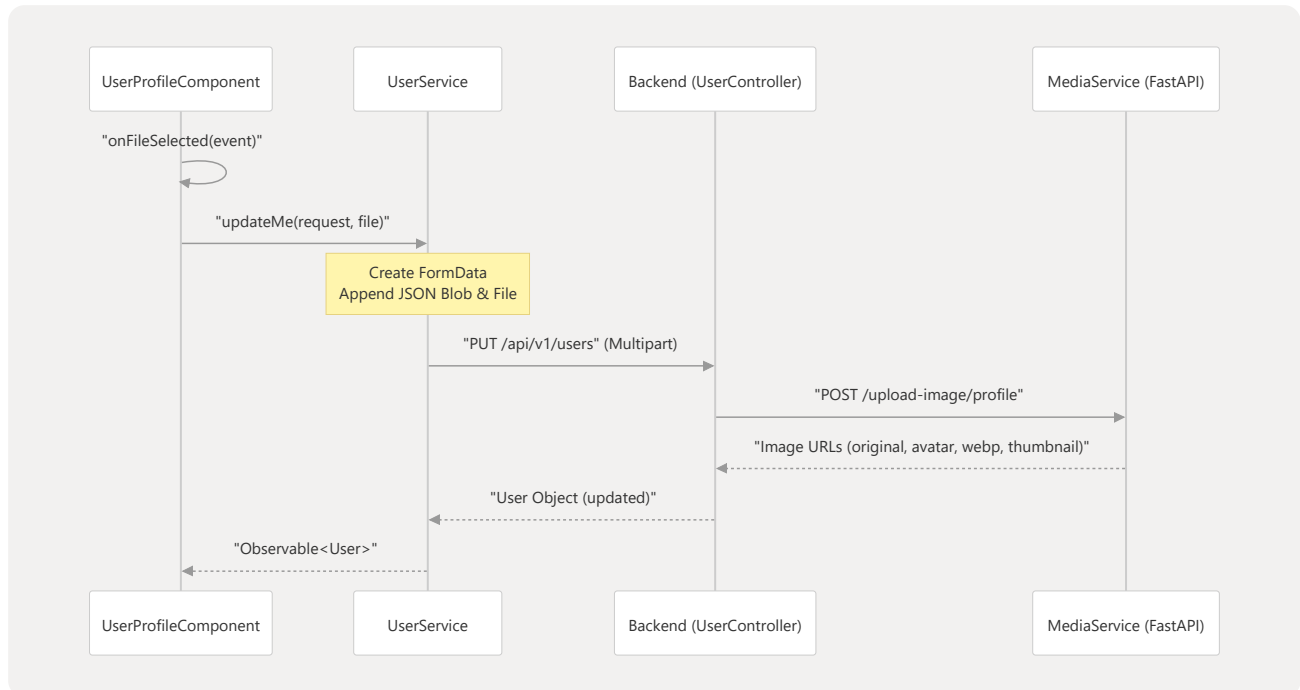
Key Functions

- `getMe()` : Fetches the private profile of the currently authenticated user
Frontend/STUDER/src/app/features/users/user.service.ts 16-18
- `getByUsername(username)` : Fetches public profile data using the `@username` handle
Frontend/STUDER/src/app/features/users/user.service.ts 20-22
- `searchUsers(query, page, size)` : Performs a paginated search for users
Frontend/STUDER/src/app/features/users/user.service.ts 24-27
- `updateMe(request, file)` : Updates user details and/or profile picture. It uses `FormData` to wrap a JSON blob for the `UserUpdateRequestDTO` and an optional `File`

Profile Update Flow

The following diagram illustrates the interaction between the `UserProfileComponent` and the `UserService` during a profile update involving an image.

User Profile Update Sequence



Sources: Frontend/STUDER/src/app/features/users/user.service.ts 29-36

Frontend/STUDER/src/app/features/users/components/user-profile/user-profile.component.ts

145-165

Social Graph & Friend Service

The `FriendService` manages social interactions. Relationships in STUDER are directional (following) but can become reciprocal (friendship).

Relationship Status Logic

The system uses the `FriendStatusResponseDTO` to inform the UI of the relationship state between the logged-in user and a target profile

Backend/STUDER/src/main/java/facu/studer/DTOs/user/FriendStatusResponseDTO.java 18-21

- **followUser(userId)** : Initiates a follow. If the target already follows the user, the backend promotes the relationship to **isFriend = true**

Frontend/STUDER/src/app/features/users/friend.service.ts 19-21

- **unfollowUser(userId)** : Removes the follow. This also breaks the **isFriend** status

Frontend/STUDER/src/app/features/users/friend.service.ts 23-25

- **getFriendStatus(userId)** : Checks the current relationship flags

Frontend/STUDER/src/app/features/users/friend.service.ts 31-33

Sources: Frontend/STUDER/src/app/features/users/friend.service.ts 1-40

Backend/STUDER/src/main/java/facu/studer/DTOs/user/FriendStatusResponseDTO.java 1-23

User Components

UserProfileComponent

The **UserProfileComponent** serves a dual purpose: it displays public profiles for other users and provides the editing interface for the owner.

- **Identity Detection:** It compares the requested username with the **AuthStateService** to set **isOwnProfile**

Frontend/STUDER/src/app/features/users/components/user-profile/user-profile.component.ts 75-85

- **Social Actions:** If **isOwnProfile** is false, it shows "Message" and "Follow/Unfollow" buttons

Frontend/STUDER/src/app/features/users/components/user-profile/user-profile.component.html 21-37

- **Editing:** If **isOwnProfile** is true, it displays a form to update email and password, along with a file input for the profile picture

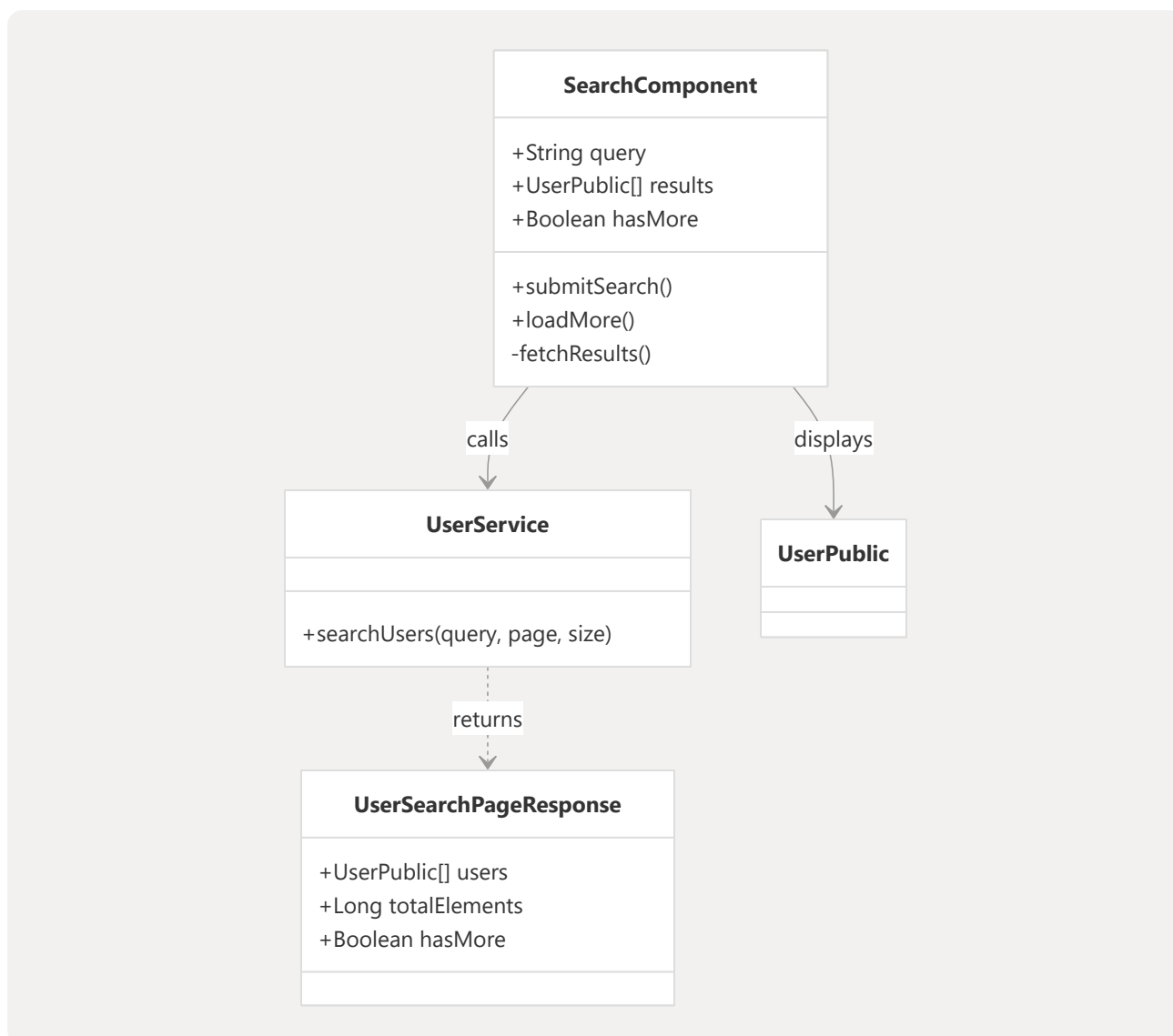
Frontend/STUDER/src/app/features/users/components/user-profile/user-profile.component.html 40-71

SearchComponent

The **SearchComponent** provides a dedicated page for user discovery.

- **Reactive Search:** It subscribes to `ActivatedRoute` query parameters (`?q=...`) to trigger searches `Frontend/STUDER/src/app/features/search/search.component.ts` 35-46
- **Pagination:** Uses a `loadMore()` function to increment the `page` counter and append results to the `results` array `Frontend/STUDER/src/app/features/search/search.component.ts` 63-67
- **UI Grid:** Displays results in a card layout using `UserPublic` data, showing the `profilePictureThumbnailUrl` or user initials if no image is present `Frontend/STUDER/src/app/features/search/search.component.html` 22-40

Search Implementation Mapping



Sources: `Frontend/STUDER/src/app/features/search/search.component.ts` 1-95

`Frontend/STUDER/src/app/features/search/search.component.html` 1-50

`Frontend/STUDER/src/app/features/users/user.service.ts` 24-27

Notification Integration

Social actions like following trigger notifications of type `USER`. These are handled by the backend's `NotificationFactory` and returned via `NotificationResponseDTO`

Backend/STUDER/src/main/java/facu/studer/DTOs/notification/NotificationResponseDTO.java 16-34

When a user clicks a follow notification, the frontend uses the `linkedId` to navigate to the follower's profile.

Sources:

Backend/STUDER/src/main/java/facu/studer/DTOs/notification/NotificationResponseDTO.java 1-34

Backend/STUDER/src/main/java/facu/studer/DTOs/notification/UserNotificationResponseDTO.java 1-34